

WindsorML Dataset Splits

Deterministic train/validation/test splits for the [WindsorML dataset](#) [1]. WindsorML contains 355 Halton-sampled variants of the Windsor body, indexed from `run_0` through `run_354`. The associated paper reports a 60/20/20 partition for its preliminary ML evaluation but does not publish an exact case-membership list [2]. For consistency with the AhmedML and DrivAerML split packages, this package instead defines an approximately 80/10/10 seed-42 baseline and companion data-efficiency and out-of-distribution (OOD) splits.

The source of truth is `splits/manifest.json`, a flat JSON object with keys such as `full_train`, `full_val`, and `geometry_test`. This repository intentionally stores only the manifest, small CSV inputs and metrics, scripts, figures, and this report. Large STL and PNG assets belong in a sibling asset directory.

Splits at a glance

Split	Type	Train	Val	Test	What it tests
<code>full</code>	In-dist	284	35	36	Seed-42 random baseline, approximately 80/10/10
<code>medium</code>	In-dist	95	35	36	Data efficiency, 1/3 of <code>full</code> training data
<code>scarce</code>	In-dist	47	35	36	Data efficiency, 1/6 of <code>full</code> training data
<code>super_scarce</code>	In-dist	8	35	36	Extreme data efficiency, 1/36 of <code>full</code> training data
<code>geometry</code>	OOD	248	36	71	STL-shape extrapolation using the top 20% local Chamfer-isolation score
<code>high_drag</code>	OOD	248	36	71	High-drag extrapolation using the top 20% fixed-reference <code>cd</code>
<code>low_drag</code>	OOD	248	36	71	Low-drag extrapolation using the bottom 20% fixed-reference <code>cd</code>
<code>image_wake</code>	OOD	248	36	71	Low-speed wake extrapolation from near-centreline and near-base velocity PNGs

Which split should I use?

- **Standard baseline:** `full`
- **Data efficiency:** compare `super_scarce`, `scarce`, `medium`, and `full`
- **Shape extrapolation:** `geometry`
- **Aerodynamic-coefficient extrapolation:** `high_drag` or `low_drag`
- **Wake-regime extrapolation:** `image_wake`

Using the committed splits

For the standard use case, read `splits/manifest.json`; no STL, PNG, or regeneration step is required. Each value is a sorted list of dataset directory names.

```
import json
from pathlib import Path

manifest = json.loads(Path("splits/manifest.json").read_text())

train_ids = manifest["full_train"]
val_ids = manifest["full_val"]
test_ids = manifest["full_test"]
```

Change the full prefix to `medium`, `scarce`, `super_scarce`, `geometry`, `high_drag`, `low_drag`, or `image_wake` to select another split.

Design principles

- **Validation remains in-distribution with training.** For each OOD split, the test set is the held-out extreme 20%; validation is the companion-package 10% sample drawn only from the complementary population.
- **Companion-package ratios.** `full` is approximately 80/10/10. OOD families are approximately 70/10/20 so the extreme test regime remains large enough to evaluate separately.
- **Nested data-efficiency subsets.** The 8-case, 47-case, and 95-case training sets are strict nested subsets of `full_train`; all share `full_val` and `full_test`.
- **Use the appropriate source quantity.** Drag splits use the constant-reference-area coefficients in `force_mom_all.csv`; geometry uses STL-surface Chamfer distance; image wake uses published streamwise-velocity PNGs.
- **Test-set integrity.** Test cases should not be used for normalization fitting, hyperparameter tuning, model selection, or repeated visual inspection during development.

Figure 1 shows the exact train, validation, and test membership against each split-defining quantity. The first panel confirms that `full` is random with respect to drag; the high- and low-drag panels show the expected coefficient tails; the lower panels show the geometry and image-wake OOD regions.

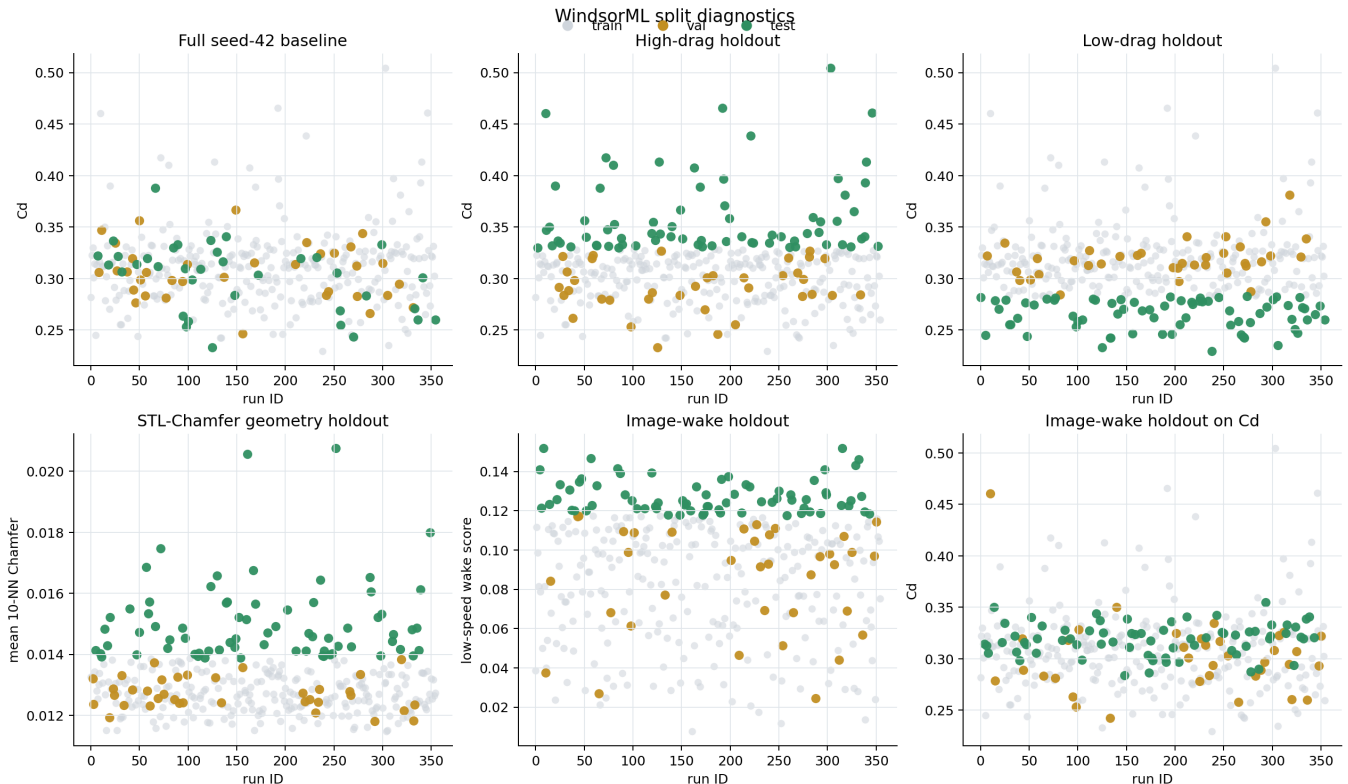


Figure 1: WindsorML split diagnostics. Points are colored by train, validation, and test membership. The panels show the full seed-42 baseline on Cd; high- and low-drag holdouts on Cd; the STL-Chamfer geometry holdout; the image-wake holdout on its low-speed score; and the image-wake membership on Cd.

Split details

`full`

To match the AhmedML and DrivAerML split convention, `full` uses an approximately 80/10/10 partition: 284 train, 35 validation, and 36 test cases. To make the unpublished membership reproducible, this repository constructs `torch.randperm(355)` with `torch.Generator().manual_seed(42)`, assigns the first 284 entries to train, the next 35 to validation, and the final 36 to test, then sorts each stored ID list. The resulting IDs are committed directly in the generator, so PyTorch is not a runtime dependency.

`medium`, `scarce`, and `super_scarce`

These families keep `full_val` and `full_test` fixed. Their training sets are nested, greedy max-min subsets of `full_train` in standardized `cd`, `cl`, and published aggregate geometry-feature space. The procedure selects a force/geometry-extreme case first, then repeatedly adds the candidate whose nearest selected neighbor is farthest away.

The WindsorML paper defines seven CAD variables: front-to-rear length ratio, back-fast length ratio, nose-to-windshield height ratio, fast-back height ratio, side taper, clearance, and bottom taper angle [2]. The current public `geo_parameters_all.csv` contains six of those variables plus `frontal_area`; `ratio_length_front_rear` appears in per-run geometry CSVs but is absent from the aggregate table. The nested selection uses the columns actually present in the aggregate file. The `geometry` OOD split does not depend on that table omission because it is computed from STLs.

`geometry`

Each available `run_N/windsor_N.stl` is sampled with 4096 deterministic, area-weighted surface points. Point clouds remain in the common dataset coordinate frame and are scaled by the global median STL bounding-box diagonal. The symmetric Chamfer RMS distance is calculated for every pair, and each run’s OOD score is its mean distance to the 10 nearest neighboring STLs. The 71 highest scores form `geometry_test`; 36 validation cases are sampled from the remaining population, leaving 248 training cases.

The current highest observed geometry scores include `run_252`, `run_161`, `run_349`, `run_72`, and `run_57`. Figure 2 compares the lowest and highest observed score examples and overlays the complete side-view geometry.

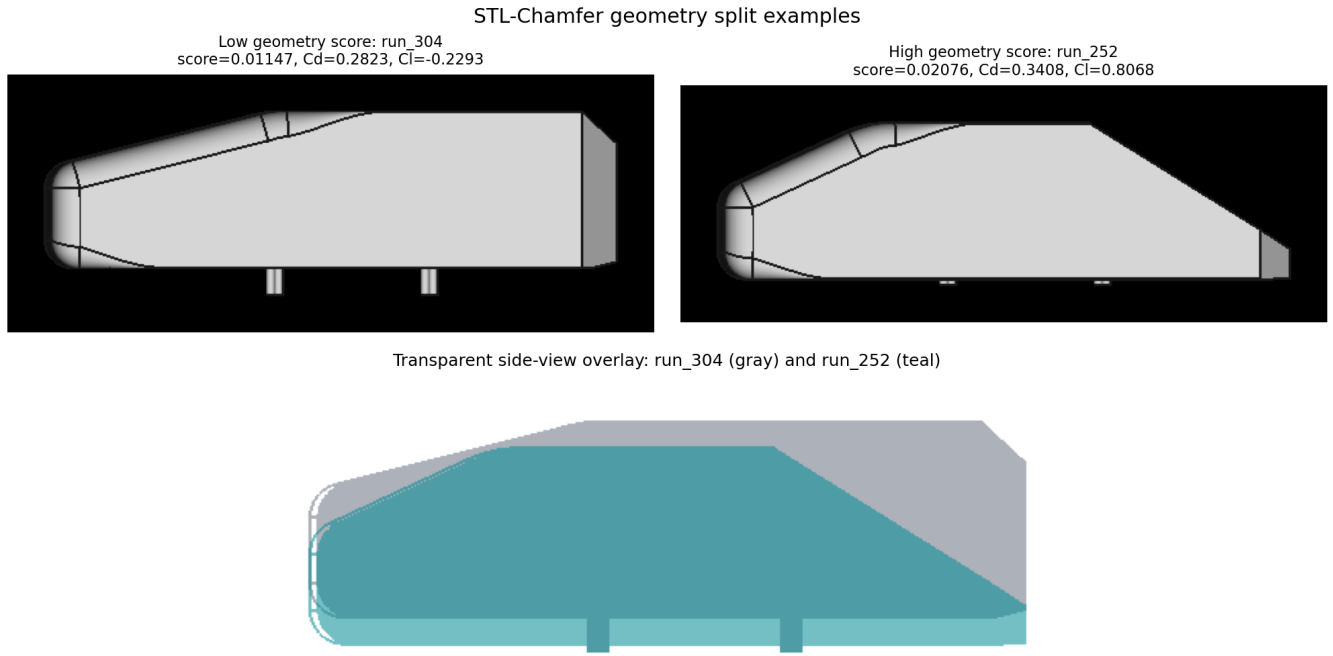


Figure 2: Geometry examples for `geometry`. The low-score case is `run_304` with Chamfer score 0.01147, $C_d=0.2823$, and $C_l=-0.2293$. The high-score case is `run_252` with score 0.02076, $C_d=0.3408$, and $C_l=0.8068$. The lower panel superimposes both complete-car side views with transparent silhouettes.

high_drag and low_drag

WindsorML publishes both constant-reference-area and case-dependent-reference-area force tables. These splits use `force_mom_all.csv`, whose constant reference area makes `cd` directly comparable across geometry variants. `high_drag` holds out the 71 largest values; `low_drag` holds out the 71 smallest. Figure 1 shows both tails.

The highest-drag runs include `run_303`, `run_192`, `run_346`, `run_10`, and `run_221`. The lowest-drag runs include `run_238`, `run_125`, `run_306`, `run_133`, and `run_134`.

image_wake

The paper states that the published images span 10 constant- z planes from $z = -0.4$ to 0.4 m and 80 constant- x planes from $x = -0.5$ to 1.0 m [2]. The image-wake score uses `view1_constz` indices 4 and 5, which bracket the centreline, and `view2_constx` indices 53, 55, and 57, immediately downstream of the Windsor base at $x = 0.48$ m. In fixed wake crops, blue/purple pixels indicate lower streamwise velocity than the orange freestream. The score averages the low-speed area and color intensity across the centreline and near-base views. The 71 largest scores form `image_wake_test`.

The highest observed image-wake scores include `run_8`, `run_315`, `run_57`, `run_332`, and `run_329`. Figure 3 shows why the score separates the selected examples: the high-score case has a substantially larger low-speed region in both views.

Image-wake split examples from streamwise-velocity PNGs

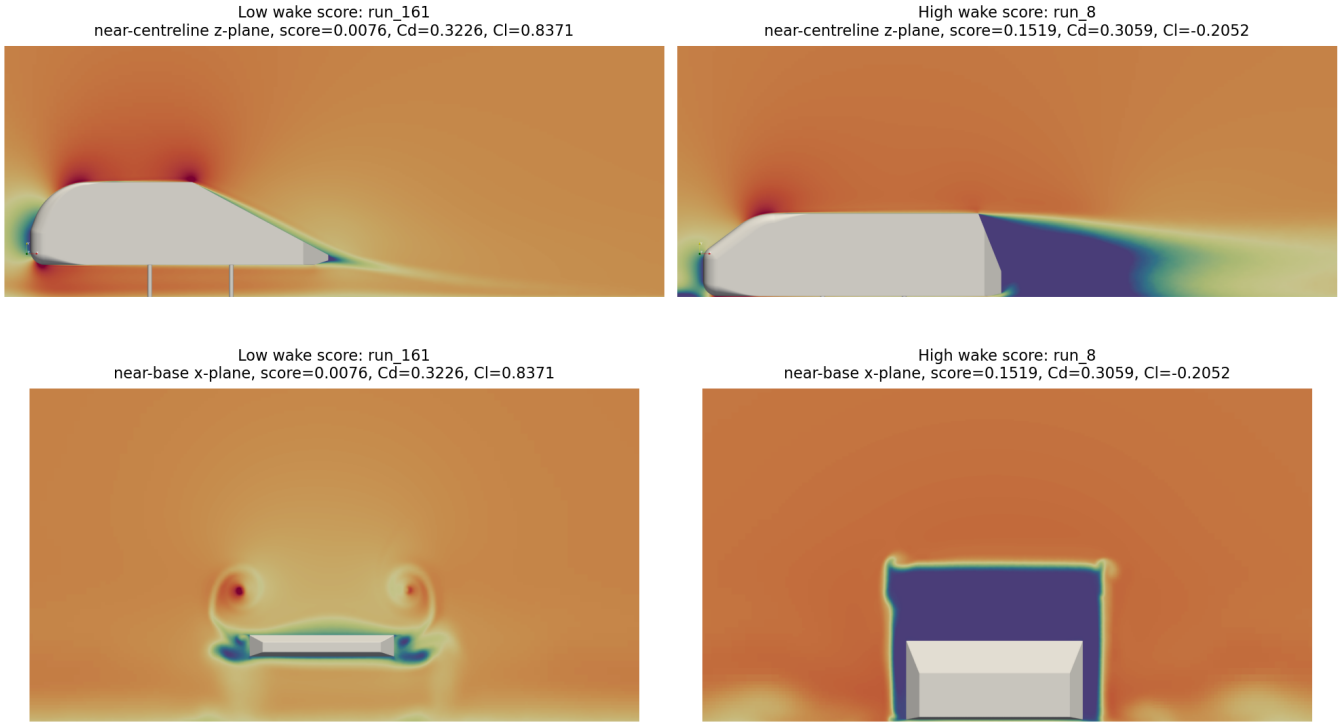


Figure 3: Image-wake examples for `image_wake`. The low-score case is `run_161` with score 0.0076, Cd=0.3226, and Cl=0.8371. The high-score case is `run_8` with score 0.1519, Cd=0.3059, and Cl=-0.2052. The top row shows a near-centreline constant- z view; the bottom row shows the near-base X-53 plane.

Current Hub asset coverage

The aggregate CSVs contain all 355 runs. At the Hub revision used for this package, per-run STLs and targeted PNGs were available for `run_0` through `run_349`; the same assets for `run_350` through `run_354` returned HTTP 404. Their five geometry and image scores are estimated from the five nearest observed runs in standardized force/aggregate-geometry space. Both `data/chamfer_metrics.csv` and `data/image_metrics.csv` include an observed flag and the neighbor IDs for every estimate. Re-running the metric scripts after those Hub files become available will replace the estimates with direct observations.

Repeatability and transparency

The committed manifest is intended for normal benchmark use. The commands below are only for auditing or rebuilding the split definitions. Keep large files in a sibling directory:

```
ASSET_ROOT=../windsorml_hf_assets

python3 scripts/download_hf_inputs.py --output-dir "$ASSET_ROOT" \
  --include-stls --include-wake-images --include-geometry-images \
  --workers 6 --allow-missing

python3 scripts/compute_chamfer_splits.py --data-root "$ASSET_ROOT" \
  --output-dir /tmp/windsorml_chamfer_4096 --samples 4096 \
  --workers 16 --sample-workers 2 --runs all --allow-missing
cp /tmp/windsorml_chamfer_4096/chamfer_metrics.csv \
  data/chamfer_metrics.csv
```

```
python3 scripts/compute_image_metrics.py --asset-root "$ASSET_ROOT" \
--data-root data --output data/image_metrics.csv
python3 scripts/generate_splits.py
python3 scripts/visualize_splits.py
python3 scripts/create_example_figures.py --asset-root "$ASSET_ROOT"
latexmk -pdf -cd docs/README.tex
```

The Chamfer script requires `numpy`, `scipy`, and `trimesh`; the image and figure scripts require `numpy`, `Pillow`, and `matplotlib`. The repository's `data/` directory should remain limited to small CSV files.

The committed source artifacts are:

```
data/force_mom_all.csv
data/geo_parameters_all.csv
data/chamfer_metrics.csv
data/image_metrics.csv
<asset-root>/run_*/windsor_*.stl
<asset-root>/run_*/images/windsor_*.png
<asset-root>/run_*/images/velocityxavg/*.png
docs/split_diagnostics.png
docs/geometry_score_examples.png
docs/wake_score_examples.png
splits/manifest.json
```

Manifest format

```
{
  "full_train": ["run_0", "run_1", "..."],
  "full_val": ["run_8", "..."],
  "full_test": ["run_7", "..."],
  "geometry_train": ["run_0", "..."]
}
```

Case IDs match the on-disk dataset directory names and are sorted numerically.

References

- [1] WindsorML dataset. <https://huggingface.co/datasets/neashton/windsorml>.
- [2] Ashton, N. et al. "WindsorML: High-Fidelity Computational Fluid Dynamics Dataset for Automotive Aerodynamics." <https://arxiv.org/abs/2407.19320>.